

面向 Ansys Maxwell 的工业软件智能体原型

自然语言驱动的仿真规格生成、软件执行与反馈修正

阶段汇报精简版

2026 年 4 月 29 日

摘要

本项目以 Ansys Maxwell 为对象，构建了一个工业软件智能体原型。用户输入自然语言需求后，系统将需求整理为仿真规格和执行计划，生成 PyAEDT 脚本，在本地调用 Maxwell 完成建模、求解和工程文件保存，并对仿真结果进行约束判定。当关键约束不满足时，系统会把结果和失败原因反馈给大模型，由大模型生成下一轮修改方案，再重新仿真验证。当前系统已验证二维电磁铁、二维平行板电容器、二维变压器概念模型和二维电感器概念模型四类任务，能够生成真实的 .aedt 工程文件和结构化结果。

关键词：工业软件智能体；Ansys Maxwell；PyAEDT；大模型；仿真自动化

1 问题与目标

工业仿真软件的难点不只是操作复杂，更在于用户需要把工程语言转换成软件可执行的建模、激励、边界和求解流程。例如“做一个 24V 直流电磁铁，气隙 2mm，电流不超过 2A”这句话，对软件来说并不是一个可直接执行的任务。它需要先被转换为几何参数、线圈参数、材料、激励、求解器设置和结果判定规则。

本项目的目标是打通一条可执行链路：

自然语言需求 → 仿真规格 → PyAEDT 脚本 → Maxwell 求解 → .aedt 工程与结果判定

这里的大模型不是直接生成 .aedt 文件，而是负责需求理解、仿真规格生成和失败后的设计修正；本地执行器负责脚本校验、Maxwell 调用、结果提取和约束判断。这样既利用了大模型的理解能力，也保留了工业软件执行的不确定性。

2 方法

2.1 统一中间表示

系统先把用户需求整理成统一的仿真规格。仿真规格可以表示为：

$$\Sigma = \{G, M, E, B, S, Q, C\}$$

其中 G 表示几何, M 表示材料, E 表示激励, B 表示边界条件, S 表示求解器, Q 表示需要提取的结果, C 表示用户提出的约束。随后系统生成执行计划:

$$P = \{a_1, a_2, \dots, a_n\}$$

每个 a_i 对应一个 Maxwell 操作, 例如创建几何、赋材料、施加电流或电压、建立求解设置、运行求解和导出结果。最终执行计划被转换为 PyAEDT 脚本, 由本地 Maxwell 执行并保存 `.aedt` 工程。

2.2 脚本执行与安全控制

自动生成脚本存在不稳定风险。系统会先做静态检查, 确认脚本具备入口函数、Maxwell 调用和工程保存逻辑, 同时限制危险操作。脚本不会在主进程中直接执行, 而是在独立 Python 进程中运行。这样即使脚本或 Maxwell 退出异常, 也不会拖死主程序。

Student 版 Maxwell 对并发较敏感, 因此网页端采用单任务运行策略, 避免多个仿真任务同时启动造成工程锁冲突或进程残留。

2.3 结果判定与反馈修正

系统不把“Maxwell 跑完”直接等同于“需求满足”。每次仿真后, 系统会读取输出并逐条判断约束。以电磁铁为例, 若用户要求 24V 且电流不超过 2A, 则系统会估算线圈电阻 R_{coil} , 并计算:

$$I_{\text{est}} = \frac{V}{R_{\text{coil}}}$$

只有当 $I_{\text{est}} \leq I_{\text{max}}$ 时, 才认为供电电压和电流约束同时满足。若不满足, 系统会把当前结果、失败原因和用户原始需求反馈给大模型, 让大模型生成下一轮修改方案, 再重新仿真。

3 实现效果

当前系统已验证四类 Maxwell 任务族。表 1 给出最新实验结果。

表 1: 已验证任务与实验结果

任务	关键结果	评价
二维电磁铁	最大磁密 9.866×10^{-5} T; 线圈电阻 13.41 Ω ; 24V 下估算电流 1.79 A, 满足不超过 2A 的约束	通过
二维平行板电容器	电容 198.18 pF; 最大电场约 3.70×10^5 V/m	通过
二维变压器概念模型	原边 10000 V, 副边目标 220 V; 匝比 0.022; 估算副边电压 220 V	通过
二维电感器概念模型	估算电感 0.326 H; 最大磁密 9.029×10^{-5} T	通过

其中，电磁铁任务已经验证了反馈修正能力。首轮设计若导致 24V 下电流超过上限，系统会把失败原因反馈给大模型，第二轮调整线圈设计后重新运行，最终得到满足电压和电流联合约束的方案。

4 复现方式

复现实验需要本地安装 Ansys Maxwell / AEDT，并配置 Python 3.12 环境和云端模型 API。项目根目录为：

```
F:\maxwell_agent_project
```

首先检查 Maxwell 和大模型接口：

```
.\.venv\Scripts\python.exe -m maxwell_agent.cli probe-env  
.\.venv\Scripts\python.exe -m maxwell_agent.cli smoke-llm
```

运行电磁铁实验：

```
.\.venv\Scripts\python.exe -m maxwell_agent.cli run "做一个24V直流电磁铁，气隙2mm，电流不超过2A，尽量提高吸力，外形不要太大。"
```

运行电容器实验：

```
.\.venv\Scripts\python.exe -m maxwell_agent.cli run "做一个二维平行板电容器，板间距1mm，板宽20mm，施加100V电压，求电场强度和电容。"
```

每次运行都会在 workspace 下生成独立目录。判断是否成功主要看三个文件：

- *.aedt: Maxwell 工程文件是否生成。
- outputs.json: 仿真结果是否导出。
- evaluation.json: 总体评价是否为 passed。

5 局限与下一步

当前系统已经不是单一电磁铁脚本，但仍然是原型阶段。稳定验证的主要是四类二维或概念模型，尚不能覆盖任意 Maxwell 问题。变压器和电感器目前偏概念验证，还没有完整覆盖损耗、温升、复杂绕组、三维结构和瞬态耦合等工程细节。

下一步需要继续扩展 Maxwell 操作原语、后处理能力和反馈修正策略。重点不是继续增加演示页面，而是让更多任务能够从自然语言稳定落到真实 Maxwell 工程，并让系统对“哪些条件满足、哪些条件不满足”给出更可靠的工程判断。

6 结论

本项目完成了一条从自然语言到 Maxwell 仿真执行结果的智能体链路。系统能够生成真实 .aedt 工程文件，提取仿真结果，并对用户的数值约束进行判断。实验结果表明，该路线能够支持多类 Maxwell 任务，并能通过“仿真反馈给大模型再修正”的方式处理部分约束冲突。当前成果适合作为工业软件智能化项目的阶段性原型，后续可在任务覆盖范围和工程精度上继续扩展。